SQL İndekslər və Performans Optimizasiyası

Dərin Nəzəri və Praktiki Bələdçi

📋 Mündəricat

1. İndeks Nədir və Necə İşləyir
2. İndeks Növləri (B-Tree, Hash, GiST, GIN, BRIN)
3. Nə Vaxt İndeks Lazımdır
4. EXPLAIN və EXPLAIN ANALYZE
5. Query Optimization Strategiyaları
6. İndeks Strategiyaları
7. Clustered vs Non-clustered İndekslər
8. Praktiki Nümunələr və Case Study
9. Performance Tuning Best Practices

1. İndeks Nədir və Necə İşləyir?

İndeks - verilənlər bazasında məlumatların sürətli axtarışı üçün yaradılan xüsusi data strukturudur. Kitabın arxasındaki indeks kimi, SQL indeksi də məlumatın harada olduğunu göstərir və axtarışı sürətləndirir.

> 💡 **Nümunə:**
>
> Kitabxanada kitab axtarırıq. İki yol var:
> 1.
> **İndeks olmadan:**
>
> Hər rəfi, hər kitabı tək-tək yoxlayırıq (Table Scan)
> 2.
> **İndeks ilə:**
>
> Kataloqa baxıb dərhal lazımi rəfə gedirik (Index Scan)

> 💡 **Nümunə:**
>
> Kitabxanada 100,000 kitab var. Müəllif adına görə kitab axtarırsınız:
>
> **İndeks olmadan:**
>
> Hər rəfə gedib, hər kitabı açıb müəllif adını yoxlayırsınız. 2-3 saat çəkər.
>
> **İndeks ilə:**
>
> Kataloqa (indeksə) baxırsınız: "Dostoyevski → Rəf 45, Sıra 12". Birbaşa gedib kitabı götürürsünüz. 2 dəqiqə çəkər.

1.1. İndeksin İş Prinsipi

SQL verilənlər bazasında məlumat **heap** adlanan strukturda saxlanılır - sətir əlavə olunduqca ardıcıl yazılır. Əgər 1 milyon sətirli cədvəldə email="test@example.com" olan istifadəçini axtarsaq:

```
-- İndeks olmadan:
SELECT * FROM users WHERE email = 'test@example.com';

-- Nə baş verir?
-- 1. Sətir 1-i oxu: email yoxla → uyğun deyil
-- 2. Sətir 2-ni oxu: email yoxla → uyğun deyil
-- 3. Sətir 3-ü oxu: email yoxla → uyğun deyil
-- ...
-- 750,000. Sətir 750000-i oxu: email yoxla → uyğundur! ✅
-- Vaxt: 750,000 disk oxuma əməliyyatı ≈ 2.5 saniyə
```

```sql
-- İndeks ilə:
CREATE INDEX idx_users_email ON users(email);
SELECT * FROM users WHERE email = 'test@example.com';

-- Nə baş verir?
-- 1. İndeksə bax (B-Tree strukturu)
-- 2. Binary search ilə 'test@example.com' tap
-- 3. İndeks pointer göstərir: Səhifə 1458, Sətir 12
-- 4. Birbaşa həmin sətri oxu
-- Vaxt: log₂(1,000,000) ≈ 20 əməliyyat ≈ 0.003 saniyə
-- 833x daha sürətli! 🚀
```

1.1. İndeksin Daxili Strukturu

İndeks iki əsas komponentdən ibarətdir:

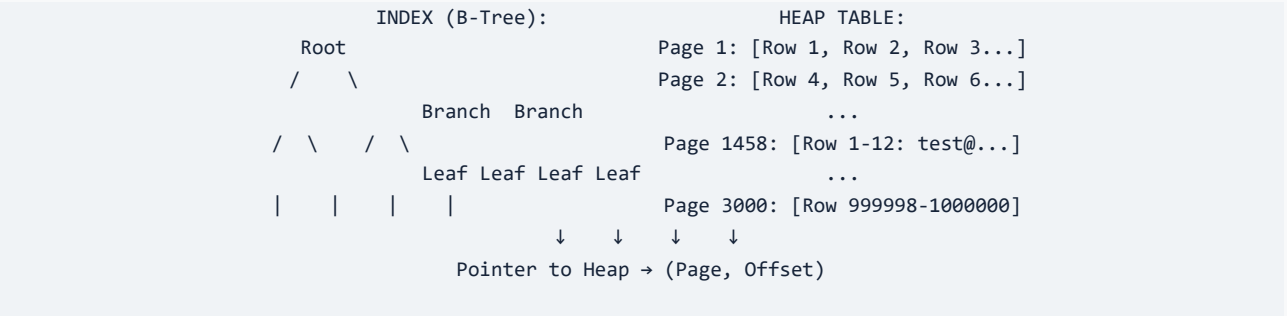
- **Search Key:** İndeksləşdirilən sütun(lar)ın dəyərləri
- **Data Pointer:** Həmin dəyərin cədvəldə yerləşdiyi ünvan (page ID, row ID)

1.2. İndeksin Daxili Strukturu

İndeks iki əsas hissədən ibarətdir:

Komponent	Təyinatı	Nümunə
Search Key	İndeksləşdirilən sütunun dəyəri	'ali@example.com'
Pointer (CTID)	Həmin dəyərin cədvəldə fiziki ünvanı	(page=1458, offset=12)

📊 İndeks və Heap Əlaqəsi



1.3. İndeks Yaratmaq və Silmək

```
-- Sadə index yaratmaq
CREATE INDEX idx_users_email ON users(email);

-- Unique index (dublikat olmaz)
CREATE UNIQUE INDEX idx_users_email_unique ON users(email);

-- Composite index (çoxsütunlu)
CREATE INDEX idx_users_name ON users(lastname, firstname);

-- Partial index (şərtli)
CREATE INDEX idx_active_users ON users(email) WHERE is_active = TRUE;

-- Functional index (funksiyanı indeksləşdir)
CREATE INDEX idx_users_email_lower ON users(LOWER(email));

-- Index silmək
DROP INDEX idx_users_email;

-- Index məlumatlarını yeniləmək
REINDEX INDEX idx_users_email;
```

```
-- Sadə index yaratmaq
CREATE INDEX idx_users_email
ON users(email);

-- Bu nə edir?
-- 1. users cədvəlindəki bütün email dəyərlərini götürür
-- 2. Onları sıralayır (sort edir)
-- 3. Hər email üçün pointer (ünvan) saxlayır
-- 4. Ayrıca data struktur (B-Tree) yaradır
```

1.4. İndeks Olmadan vs İndeks İlə Axtarış

🔍 Sequential Scan (İndeks Yoxdur)

Table: [Row1] → [Row2] → [Row3] → ... → [Row 1,000,000]

Vaxt: O(n) - Hər sətri yoxlamaq lazımdır

Index Scan (İndeks Var)

B-Tree İndeks → Sürətli axtarış → Birbaşa lazımi sətrə keçid

Vaxt: O(log n) - Yalnız ağac səviyyələrini keçmək lazımdır

İndeks Olmadan	İndeks İlə	Sürətlənmə
2.5 san	**0.003 san**	**833x**
1M sətirli cədvəldə axtarış	1M sətirli cədvəldə axtarış	Performans artımı

2. İndeks Növləri - Dərin Təhlil

2.1. B-Tree İndeks (Default və Ən Populyar)

B-Tree (Balanced Tree) - özünü balanslayan ağac strukturudur. SQL-də default index tipidir.

> **B-Tree Strukturu:**
>
> B-Tree çoxsəviyyəli ağacdır. Hər node (qovşaq) çoxlu key saxlayır və sıralı saxlanılır.

B-Tree Necə İşləyir?

```
-- Məsələn: 1 milyon istifadəçi, ID ilə axtarış
SELECT * FROM users WHERE id = 750000;

-- B-Tree İndeks olduqda:
-- Səviyyə 1 (Root):     [500000]
--                      /        \
-- Səviyyə 2:   [250000]      [750000]
--                            /    \
-- Səviyyə 3:            [625000] [875000]
--                         /
-- Səviyyə 4 (Leaf):   [749999, 750000, 750001...]

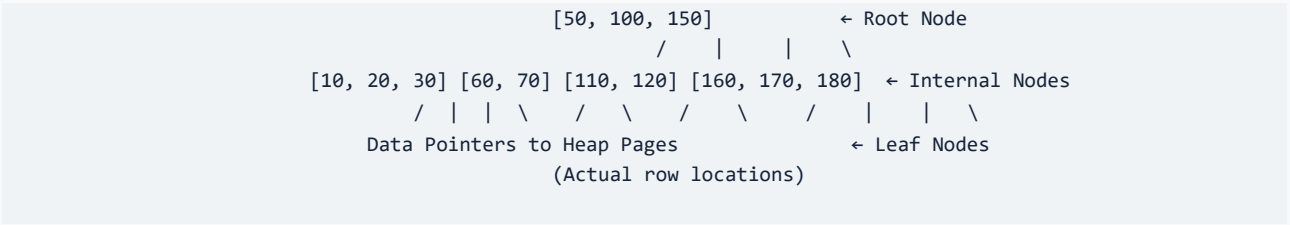
-- Yalnız 4 addımda tapırıq (log₂ 1,000,000 ≈ 20 addım)
```

B-Tree (Balanced Tree) SQL-də ən geniş yayılmış və default index tipidir. "Balanced" sözü burada çox vacibdir - ağac həmişə balanslaşdırılır, yəni hər leaf node eyni dərinlikdədir.

2.1.1. B-Tree Strukturu və İş Prinsipi

B-Tree çoxsəviyyəli ağac strukturudur. Hər node çoxlu key saxlayır və child pointer-lər var. PostgreSQL-də default B-Tree order = 128 (hər node-da max 128 key).

🌳 B-Tree Strukturu (Sadələşdirilmiş)



Əsas Prinsip: Hər node-da key-lər sıralıdır. Sol child < key < Sağ child

```
-- Gəlin 1 milyon sətirli cədvəldə ID axtarışı edək
CREATE TABLE users (
    id SERIAL PRIMARY KEY,  -- Avtomatik B-Tree index yaradır
    email VARCHAR(255),
    name VARCHAR(100)
);

-- 1M sətir əlavə edək
INSERT INTO users (email, name)
SELECT
    'user' || i || '@example.com',
    'User ' || i
FROM generate_series(1, 1000000) AS i;

-- İndi ID ilə axtaraq
SELECT * FROM users WHERE id = 750000;

-- B-Tree necə işləyir:
-- 1. Root node-a get: [1, 250000, 500000, 750000, 1000000]
--    750000 axtarırıq → 4-cü bölməyə get
-- 2. Internal node: [750000, 812500, 875000, 937500]
--    750000 axtarırıq → 1-ci bölməyə get
-- 3. Leaf node: [750000] → Pointer: (page=1458, offset=12)
-- 4. Heap page 1458-i oxu, offset 12-dəki sətri gətir
-- Cəmi 4 disk I/O! (Sequential scan olsaydı 750,000 I/O)
```

B-Tree Üstünlükləri:

- ✅ Equality axtarış: `WHERE id = 100`
- ✅ Range axtarış: `WHERE age BETWEEN 20 AND 30`
- ✅ Ordering: `ORDER BY name`
- ✅ Pattern matching (prefix): `WHERE name LIKE 'Ali%'`
- ✅ MIN/MAX funksiyaları sürətli işləyir

```
-- B-Tree index yaratmaq (default)
CREATE INDEX idx_users_lastname ON users(lastname);

-- və ya explicit
CREATE INDEX idx_users_lastname ON users USING btree(lastname);

-- Composite B-Tree index
CREATE INDEX idx_users_name ON users(lastname, firstname);

-- Bu index bu sorğular üçün işləyir:
WHERE lastname = 'Mammadov'  -- ✅ İşləyir
WHERE lastname = 'Mammadov' AND firstname = 'Ali'  -- ✅ İşləyir
WHERE firstname = 'Ali'  -- ❌ İşləməz (left-most rule)
```

> ⚠️ **B-Tree Məhdudiyyəti:**
>
> Composite index-də sütunların sırası vacibdir! (lastname, firstname) indexi firstname-lə başlayan sorğu üçün istifadə edilə bilməz.

2.2. Hash İndeks

Hash İndeks - hash funksiyası istifadə edərək məlumatı saxlayır. Yalnız equality (=) sorğuları üçün effektivdir.

> 🔢 **Hash Funksiyası:**
>
> Hash funksiyası hər dəyərə unique (və ya az collision olan) nömrə verir. Məsələn: hash('ali@example.com') = 34567

Hash İndeks Üstünlükləri:

- ✅ Equality axtarış çox sürətli: O(1)
- ✅ Daha az disk space tutur
- ✅ Memory cache üçün ideal

Hash İndeks Məhdudiyyətləri:

- ❌ Range query işləməz: `WHERE age > 25`
- ❌ LIKE işləməz: `WHERE name LIKE 'Ali%'`
- ❌ ORDER BY istifadə edə bilməz
- ❌ MIN/MAX slow

```sql
-- Hash index yaratmaq (PostgreSQL)
CREATE INDEX idx_users_email_hash
ON users USING hash(email);

-- Bu sorğu üçün perfect:
SELECT * FROM users WHERE email = 'ali@example.com';  -- ⚡ O(1)

-- Bu sorğular üçün faydasız:
SELECT * FROM users WHERE email LIKE 'ali%';  -- ❌ Full scan
SELECT * FROM users ORDER BY email;  -- ❌ Sort lazım
```

📌 Nə Vaxt Hash İstifadə Etmək Lazımdır?

- Yalnız equality axtarışlar (=) edirsiniz
- Unique identifier-lər (UUID, hash codes)
- Session keys, API tokens
- Lookup tables

2.3. GiST İndeks (Generalized Search Tree)

GiST - universal index strukturudur, müxtəlif data tipləri üçün custom axtarış yaratmağa imkan verir.

GiST İstifadə Sahələri:

- 🗺️ **Geometric data:** Points, polygons, circles
- 🔍 **Full-text search:** Text axtarış
- 🌐 **IP address ranges:** Network axtarışları
- 🗓️ **Range types:** Date/time ranges

```sql
-- Geometric data üçün GiST
CREATE TABLE locations (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    coordinates POINT
);

CREATE INDEX idx_locations_coords
ON locations USING gist(coordinates);

-- Yaxın yerləri tap
SELECT name
FROM locations
WHERE coordinates <@ circle '<(0,0),10>';

-- IP range axtarışı
CREATE TABLE ip_ranges (
    range inet
);

CREATE INDEX idx_ip_ranges ON ip_ranges USING gist(range);

SELECT * FROM ip_ranges WHERE range >> inet '192.168.1.5';
```

2.4. GIN İndeks (Generalized Inverted Index)

GIN - əsasən composite values (array, JSON, full-text) üçün istifadə olunur.

GIN İstifadə Sahələri:

- 📑 **Full-text search:** Mətn axtarışı
- 🏷️ **Array columns:** Array elementlərində axtarış
- 📄 **JSONB:** JSON data axtarışı
- 🔗 **Many-to-many relationships:** Tag sistemləri

```sql
-- Array column üçün GIN
CREATE TABLE posts (
    id SERIAL PRIMARY KEY,
    title TEXT,
    tags TEXT[]
);

CREATE INDEX idx_posts_tags ON posts USING gin(tags);

-- Tag-a görə axtarış - çox sürətli
SELECT * FROM posts WHERE tags @> ARRAY['sql', 'database'];

-- JSONB üçün GIN
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    data JSONB
);

CREATE INDEX idx_users_data ON users USING gin(data);

SELECT * FROM users WHERE data @> '{"country": "Azerbaijan"}';

-- Full-text search
CREATE INDEX idx_posts_search
ON posts USING gin(to_tsvector('english', title));

SELECT * FROM posts
WHERE to_tsvector('english', title) @@ to_tsquery('database & optimization');
```

> ✅ **GIN Performance:**
>
> GIN index böyük ola bilər, amma axtarış çox sürətlidir. Write əməliyyatları yavaş ola bilər (index update).

2.5. BRIN İndeks (Block Range Index)

BRIN - çox böyük cədvəllər üçün kiçik footprint olan indexdir. Physical storage order istifadə edir.

> 💾 **BRIN Prinsipi:**
>
> BRIN hər sətr üçün deyil, hər block (page) üçün min/max dəyərləri saxlayır. Çox kiçik, amma yalnız sorted data üçün effektiv.

BRIN Üstünlükləri:

- ✅ Çox kiçik index size (B-Tree-dən 100x-1000x kiçik)

- ✅ İnsert/Update sürətli
- ✅ Böyük time-series data üçün ideal

```
-- Time-series data üçün BRIN
CREATE TABLE sensor_data (
    id BIGSERIAL PRIMARY KEY,
    sensor_id INTEGER,
    temperature DECIMAL(5,2),
    recorded_at TIMESTAMP
);

-- BRIN index - çox kiçik, recorded_at naturally sorted olduğu üçün
CREATE INDEX idx_sensor_recorded
ON sensor_data USING brin(recorded_at);

-- Range query effektiv
SELECT AVG(temperature)
FROM sensor_data
WHERE recorded_at BETWEEN '2024-01-01' AND '2024-01-31';
```

> ⚠️ **BRIN Məhdudiyyəti:**
>
> BRIN yalnız physically sorted data üçün yaxşıdır. Random insert-lər olan cədvəllər üçün effektiv deyil.

2.6. İndeks Növləri Müqayisəsi

İndeks Tipi	İstifadə Sahəsi	Equality	Range	ORDER BY	Size
B-Tree	Universal, default	✅ O(log n)	✅ Yaxşı	✅ Yaxşı	Orta
Hash	Equality only	✅ O(1)	❌ Yox	❌ Yox	Kiçik
GiST	Geo, custom types	✅ Var	✅ Var	⚠️ Dəstək var	Böyük
GIN	Array, JSON, Text	✅ Yaxşı	⚠️ Məhdud	❌ Yox	Çox böyük
BRIN	Time-series, sorted	⚠️ Orta	✅ Yaxşı	⚠️ Orta	Çox kiçik

3. Nə Vaxt İndeks Lazımdır?

İndeks performance üçün çox vacibdir, amma həmişə lazım deyil. Düzgün qərar vermək üçün bir neçə faktor nəzərə almalıyıq.

3.1. İndeks Yaratmaq Lazım Olan Hallar

✅ **İndeks Mütləq Lazımdır:**

- **WHERE clauselə tez-tez axtarılan sütunlar**
- **JOIN əməliyyatlarında istifadə olunan sütunlar**
- **Foreign key sütunları**
- **ORDER BY və GROUP BY sütunları**
- **Böyük cədvəllər (100K+ rows)**
- **Yüksək read/write ratio (10:1 və daha çox)**

```sql
-- ✅ YAXŞI: email ilə tez-tez axtarırıq
CREATE INDEX idx_users_email ON users(email);
SELECT * FROM users WHERE email = 'test@example.com';

-- ✅ YAXŞI: JOIN üçün lazım
CREATE INDEX idx_orders_user_id ON orders(user_id);
SELECT * FROM orders o
JOIN users u ON o.user_id = u.id;

-- ✅ YAXŞI: Sıralama üçün
CREATE INDEX idx_posts_created ON posts(created_at DESC);
SELECT * FROM posts ORDER BY created_at DESC LIMIT 10;
```

3.2. İndeks Lazım Olmayan Hallar

❌ **İndeks Yaratmayın:**

- **Kiçik cədvəllər (<1000 rows)**

- Full scan sürətlidir

- **Yüksək write/read ratio**

  - Hər insert/update indexi yeniləyir

- **Low cardinality sütunlar**

  - Az unique dəyər (gender, boolean)

- **Nadir istifadə olunan sütunlar**

  - Ayda 1 dəfə axtarış

- **Çox kiçik string-lər**

  - 2-3 simvol

```
-- ❌ PISS: Boolean sütun, index faydasız
CREATE INDEX idx_users_is_active ON users(is_active);  -- Lazım deyil!

-- 50% true, 50% false olsa index istifadə edilməz
SELECT * FROM users WHERE is_active = TRUE;  -- Full scan daha sürətli

-- ❌ PISƏ: Kiçik cədvəl
CREATE TABLE countries (id INT, name VARCHAR(100));  -- 195 rows
CREATE INDEX idx_countries_name ON countries(name);  -- Lazım deyil!
```

3.3. Cardinality və Selectivity

Cardinality - sütundakı unique dəyərlərin sayı.

Selectivity - unique dəyərlərin ümumi sətirə nisbəti.

```
-- Cardinality yoxlama
SELECT
    COUNT(DISTINCT email) as unique_emails,
    COUNT(*) as total_rows,
    (COUNT(DISTINCT email)::float / COUNT(*)::float) as selectivity
FROM users;

-- Selectivity > 0.5 → Index effektivdir
-- Selectivity < 0.1 → Index faydasız ola bilər
```

📊 Selectivity Nümunələri

- **email (unique):** Selectivity = 1.0 → ✅ Əla index candidate
- **lastname:** Selectivity = 0.6 → ✅ Yaxşı index
- **country:** Selectivity = 0.001 → ⚠️ Zəif index
- **gender (M/F):** Selectivity = 0.00002 → ❌ Index faydasız

3.4. Index Overhead (İndeksin Qiyməti)

İndeks yaratmaq pulsuz deyil. Hər index bu xərcləri gətirir:

- 💾 **Disk space:** İndeks ayrıca saxlanılır (5-20% əlavə)
- ⏱️ **INSERT slow:** Hər insert indexi yeniləyir
- ⏱️ **UPDATE slow:** İndekslənmiş sütun dəyişərsə, index update
- ⏱️ **DELETE slow:** İndeksdən də silinməli
- 🧠 **Memory:** Active indexes RAM-də cache olunur

```
-- İndeks size yoxlama (PostgreSQL)
SELECT
    tablename,
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_indexes
JOIN pg_class ON pg_indexes.indexname = pg_class.relname
WHERE schemaname = 'public'
ORDER BY pg_relation_size(indexrelid) DESC;
```

4. EXPLAIN və EXPLAIN ANALYZE - Query Plan Təhlili

EXPLAIN - SQL-in query plan göstərmə əmridir. Bu komanda sorğunun necə icra olunacağını göstərir.

4.1. EXPLAIN Əsasları

> 💡 **EXPLAIN nə göstərir?**
>
> - Hansı scan metodu istifadə olunur (Seq Scan, Index Scan)
> - Hansı indexlər istifadə olunur
> - JOIN strategiyası
> - Estimated cost və rows
> - Sort, Aggregate əməliyyatları

```
-- Sadə EXPLAIN
EXPLAIN
SELECT * FROM users WHERE email = 'ali@example.com';

-- Nəticə (index yoxdursa):
-- Seq Scan on users  (cost=0.00..1234.56 rows=1 width=100)
--   Filter: (email = 'ali@example.com')

-- Nəticə (index varsa):
-- Index Scan using idx_users_email on users  (cost=0.42..8.44 rows=1 width=100)
--   Index Cond: (email = 'ali@example.com')
```

4.2. Scan Metodları

4.2.1. Sequential Scan (Seq Scan)

Cədvəlin hər sətrini oxuyur. Index yoxdursa və ya index istifadə etmək səmərəli deyilsə.

```
EXPLAIN SELECT * FROM users WHERE age > 18;

-- Seq Scan on users  (cost=0.00..1829.00 rows=48576 width=100)
--   Filter: (age > 18)
-- 48% sətir qaytarılır → Sequential scan daha yaxşı
```

4.2.2. Index Scan

İndeksdən istifadə edərək lazımi sətirlər tapılır və heap-dən oxunur.

```
CREATE INDEX idx_users_email ON users(email);

EXPLAIN SELECT * FROM users WHERE email = 'ali@example.com';

-- Index Scan using idx_users_email on users  (cost=0.42..8.44 rows=1)
--   Index Cond: (email = 'ali@example.com')
```

4.2.3. Index Only Scan

Bütün məlumat indexdə olduqda heap-ə getmədən cavab verilir.

```
CREATE INDEX idx_users_email_name ON users(email, name);

EXPLAIN SELECT email, name FROM users WHERE email = 'ali@example.com';

-- Index Only Scan using idx_users_email_name on users  (cost=0.42..4.44 rows=1)
--   Index Cond: (email = 'ali@example.com')
-- ⚡ ən sürətli - heap-ə getmir!
```

4.2.4. Bitmap Index Scan

Çoxlu sətirlər qaytarılarkən istifadə olunur. Əvvəlcə bitmap yaradır, sonra heap-i bir dəfə oxuyur.

```
EXPLAIN SELECT * FROM users WHERE age BETWEEN 25 AND 30;

-- Bitmap Heap Scan on users  (cost=100.42..1500.00 rows=5000)
--   Recheck Cond: ((age >= 25) AND (age <= 30))
--   ->  Bitmap Index Scan on idx_users_age  (cost=0.00..99.17 rows=5000)
--         Index Cond: ((age >= 25) AND (age <= 30))
```

4.3. EXPLAIN ANALYZE - Real Execution

EXPLAIN ANALYZE sorğunu həqiqətən icra edir və real statistika verir.

> ⚠️ **DİQQƏT:**
>
> EXPLAIN ANALYZE sorğunu icra edir! SELECT-də problem yoxdur, amma INSERT/UPDATE/DELETE ilə ehtiyatlı olun. Test əvvəl BEGIN, sonra ROLLBACK edin.

```
EXPLAIN ANALYZE
SELECT * FROM users WHERE email = 'ali@example.com';

-- Index Scan using idx_users_email on users
--   (cost=0.42..8.44 rows=1 width=100)
--   (actual time=0.025..0.027 rows=1 loops=1)
--   Index Cond: (email = 'ali@example.com')
-- Planning Time: 0.123 ms
-- Execution Time: 0.052 ms
```

EXPLAIN Parametrləri Oxuma:

- **cost:** Planlanan xərc (0.42..8.44 = başlanğıc..son)
- **rows:** Qaytarılacaq sətir sayı (estimated)
- **width:** Orta sətir size (byte)
- **actual time:** Real vaxt (ms)
- **loops:** Neçə dəfə icra olundu

4.4. EXPLAIN Formatları

```
-- Text format (default)
EXPLAIN SELECT * FROM users WHERE id = 1;

-- JSON format
EXPLAIN (FORMAT JSON) SELECT * FROM users WHERE id = 1;

-- Detailed format
EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
SELECT * FROM users WHERE id = 1;
```

4.5. Praktiki Nümunələr

📌 Case Study: Slow Query Optimization

Problem: 1M istifadəçidə email ilə axtarış 2.5 saniyə alır

```
-- 1. İlk vəziyyət (index yox)
EXPLAIN ANALYZE
SELECT * FROM users WHERE email = 'test@example.com';

-- Seq Scan on users  (cost=0.00..21234.00 rows=1)
--   (actual time=1.234..2478.123 rows=1 loops=1)
-- Execution Time: 2478.523 ms  ❌ Çox yavaş!

-- 2. Index yaradaq
CREATE INDEX idx_users_email ON users(email);

-- 3. Yenidən yoxlayaq
EXPLAIN ANALYZE
SELECT * FROM users WHERE email = 'test@example.com';

-- Index Scan using idx_users_email on users  (cost=0.42..8.44 rows=1)
--   (actual time=0.025..0.027 rows=1 loops=1)
-- Execution Time: 0.052 ms  ✅ 47,000x sürətli!
```

5. Query Optimization Strategiyaları

5.1. SELECT * Istifadə Etməyin

> ❌ **Pis Praktika:**
>
> SELECT * lazım olmayan sütunları da oxuyur və network traffic artırır.

```
-- ❌ PISƏ
SELECT * FROM users WHERE id = 1;
-- 50 sütun oxuyur, 500KB data

-- ✅ YAXŞI
SELECT id, name, email FROM users WHERE id = 1;
-- 3 sütun oxuyur, 0.5KB data
-- Index Only Scan istifadə edə bilər
```

5.2. WHERE Clause Optimization

```
-- ❌ PISƏ: Function sütunda
SELECT * FROM users WHERE LOWER(email) = 'test@example.com';
-- İndeks istifadə edilə bilməz!

-- ✅ YAXŞI: Function dəyərdə
SELECT * FROM users WHERE email = LOWER('TEST@EXAMPLE.COM');
-- İndeks istifadə olunur

-- ✅ Və ya functional index
CREATE INDEX idx_users_email_lower ON users(LOWER(email));
SELECT * FROM users WHERE LOWER(email) = 'test@example.com';
```

5.3. LIKE Optimization

```
-- ✅ İndeks istifadə edir (prefix search)
SELECT * FROM users WHERE lastname LIKE 'Mam%';

-- ❌ İndeks istifadə edilə bilməz (suffix search)
SELECT * FROM users WHERE lastname LIKE '%dov';

-- ❌ İndeks istifadə edilə bilməz (middle search)
SELECT * FROM users WHERE lastname LIKE '%mma%';

-- ✅ Full-text search üçün GIN index
CREATE INDEX idx_users_lastname_trgm
ON users USING gin(lastname gin_trgm_ops);

SELECT * FROM users WHERE lastname LIKE '%mma%';  -- İndi sürətli!
```

5.4. JOIN Optimization

```
-- ✅ İndeks olmalıdır
CREATE INDEX idx_orders_user_id ON orders(user_id);
CREATE INDEX idx_users_id ON users(id);  -- PK auto index

EXPLAIN ANALYZE
SELECT u.name, o.product_name
FROM users u
JOIN orders o ON u.id = o.user_id
WHERE u.id = 100;

-- Nested Loop join istifadə edəcək
-- 1. users-də id=100 tap (Index Scan)
-- 2. orders-də user_id=100 tap (Index Scan)
```

5.5. Subquery vs JOIN

```
-- ❌ PISƏ: Correlated subquery (hər sətir üçün icra olunur)
SELECT u.name,
    (SELECT COUNT(*) FROM orders o WHERE o.user_id = u.id) as order_count
FROM users u;

-- ✅ YAXŞI: JOIN və GROUP BY
SELECT u.name, COUNT(o.id) as order_count
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
GROUP BY u.id, u.name;
```

5.6. EXISTS vs IN vs JOIN

```
-- Sifariş edən istifadəçiləri tap

-- ✅ EXISTS (ən effektiv çox hallarda)
SELECT * FROM users u
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id);

-- ⚠️ IN (kiçik list üçün yaxşı)
SELECT * FROM users
WHERE id IN (SELECT user_id FROM orders);

-- ✅ JOIN (data lazımdırsa)
SELECT DISTINCT u.*
FROM users u
JOIN orders o ON u.id = o.user_id;
```

5.7. LIMIT və Pagination

```
-- ❌ PISƏ: OFFSET böyük
```